

Hadoop NameNode

核心架构设计文档

基于 release-3.3.5-RC0 版本源码深度分析

项目	Apache Hadoop HDFS
模块	hadoop-hdfs / server / namenode
版本	release-3.3.5-RC0
范围	NameNode主目录+8个子目录(ha/snapshot/metrics/sps/fgl/top/visitor/startupprogress)
文件数	~150+ Java源文件
工具	自动化源码架构分析

目录

- 一、模块概述
- 二、核心分层架构
- 三、NameNode启动流程
- 四、FSNamesystem—门面模式与核心状态机
- 五、INode层次与内存优化
- 六、HA高可用机制—状态模式
- 七、编辑日志与持久化机制
- 八、FSNamesystemLock读写锁
- 九、核心流程时序分析
- 十、设计模式总结
- 十一、关键场景调用链
- 十二、子模块概览
- 十三、核心源文件清单
- 十四、设计亮点与总结

一、模块概述

NameNode 是 HDFS 的核心元数据管理节点，负责维护整个分布式文件系统的命名空间（Namespace）。它管理文件→块映射、块→DataNode副本位置、目录树结构、权限控制、租约（Lease）、快照（Snapshot）、加密区域等所有元数据信息。模块位于 `hadoop-hdfs/server/namenode` 目录，包含约150+个Java源文件，代码总量超15万行。其中 `FSNamesystem.java`（341KB，约9200行）是HDFS中最大的单文件。

1.1 核心目标

- 命名空间管理：维护HDFS文件系统目录树（INode层次），支持文件/目录增删改查
- 块映射管理：维护文件→数据块、块→DataNode的映射关系，协调副本放置
- 元数据持久化：通过FSEditLog（编辑日志）和FSImage（文件系统镜像）实现可靠持久化
- 高可用（HA）：支持Active/Standby/Observer三种角色，实现故障自动转移
- 快照管理：支持目录级快照，实现元数据时间点回溯
- 安全与加密：Kerberos认证、ACL、加密区域、安全Token管理
- 缓存管理：DataNode缓存指令，加速热点数据访问

二、核心分层架构

NameNode模块采用清晰的四层架构设计：RPC接入层→核心业务逻辑层→数据管理层→HA+持久化层。

2.1 分层架构全景图

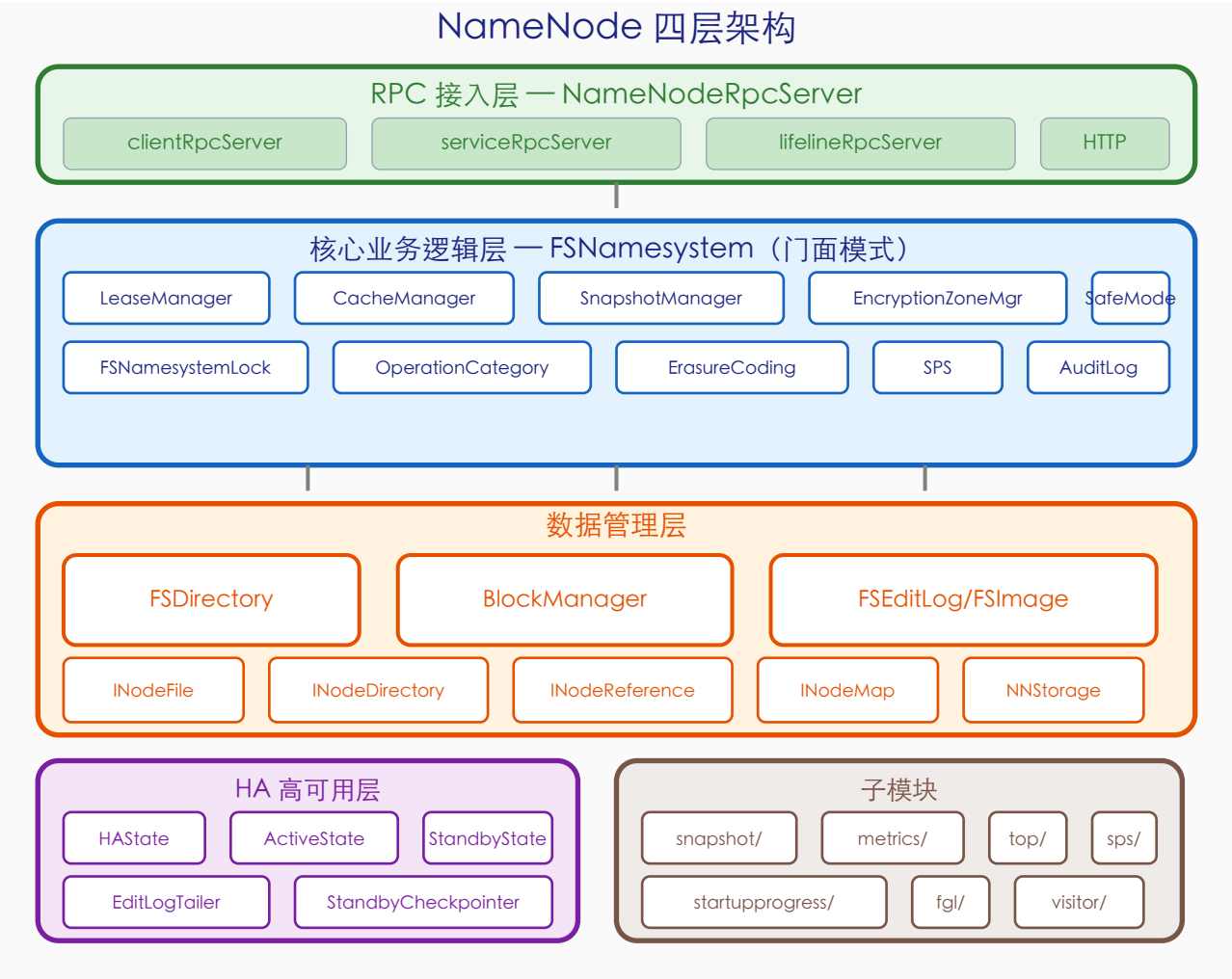


图2-1：NameNode四层架构全景图

2.2 RPC三通道设计

RPC Server	监听端口配置	职责	注册的协议
clientRpcServer	dfs.namenode.rpc-ad dress	处理客户端文件操作	ClientNamenodeProtocol等

RPC Server	监听端口配置	职责	注册的协议
serviceRpcServer	dfs.namenode.servicerpc-address	处理DataNode和NN请求	DatanodeProtocol, NamenodeProtocol, HAThrottlingProtocol
lifelineRpcServer	dfs.namenode.lifeline.rpc-address	HA健康检查专用通道	HAThrottlingProtocol(精简版)

设计意图：lifeline通道确保即使client/serviceRpc因请求积压繁忙，ZKFC仍能通过独立lifeline通道检测NameNode存活，避免不必要的HA故障转移。

三、NameNode 启动流程

入口：NameNode.main() → createNameNode() → new NameNode(conf) → initialize()。

3.1 createNameNode() — 启动选项分发

StartupOption	操作	说明
FORMAT	format(conf,...)	格式化文件系统，初始化存储目录
GENCLUSTERID	NNStorage.newClusterID()	生成新集群ID
ROLLBACK	doRollback(conf,true)	回滚到升级前版本
BOOTSTRAPSTANDBY	BootstrapStandby.run()	从Active拷贝FSImage初始化Standby
INITIALIZESHAREDEDITS	initializeSharedEdits()	初始化QJM共享编辑日志
default	new NameNode(conf)	正常启动，进入initialize()

3.2 initialize() — 核心初始化序列

步骤	方法	说明
1	loginAsNameNodeUser()	Kerberos认证登录
2	NameNode.initMetrics()	初始化指标监控和JVM暂停监控器
3	startHttpServer()	启动HTTP/Web UI服务
4	loadNamesystem(conf)	【关键】加载FSImage+重放EditLog恢复命名空间
5	createRpcServer()	【关键】创建三个RPC Server，注册所有协议
6	startCommonServices()	【关键】启动BlockManager、RPC监听、ServicePlugin

关键点：startCommonServices()内部启动BlockManager的ReplicationMonitor、HeartbeatManager、DecommissionManager等，然后rpcServer.start()开始接受外部请求。至此NameNode完全就绪。

四、FSNamesystem — 门面模式与核心状态机

FSNamesystem (341KB, ~9200行) 是NameNode核心类, 扮演门面角色 (Facade Pattern), 为NameNodeRpcServer提供统一业务入口, 内部协调FSDirectory、BlockManager、FSEditLog、LeaseManager、SnapshotManager、CacheManager、EncryptionZoneManager等子系统。

4.1 操作分类与HA检查

操作类别	允许的HA状态	典型操作
READ	Active, Observer	文件状态查询、目录列表、块位置获取
WRITE	仅Active	文件创建/删除/重命名、权限修改、块分配
CHECKPOINT	Active	SaveNamespace、滚动编辑日志
UNCHECKED	所有状态	安全Token操作等

4.2 双重锁机制

加锁模式：先获取FSNamesystem锁 (RwLockMode.FS) → 再获取FSDirectory锁 (dir.writeLock) → 避免死锁。
典型写操作：writeLock(FS) → checkOperation(WRITE) → checkSafeMode() → dir.writeLock() → 执行修改 → dir.writeUnlock() → writeUnlock(FS) → logSync()

4.3 安全模式 (SafeMode)

启动后进入安全模式, 只允许读操作。退出条件：已报告块数/总块数 >= 0.999 (默认), 加上30秒扩展等待。安全模式下所有写操作被拒绝并抛出SafeModeException。

五、文件系统树 — INode层次与内存优化

5.1 INode类继承体系

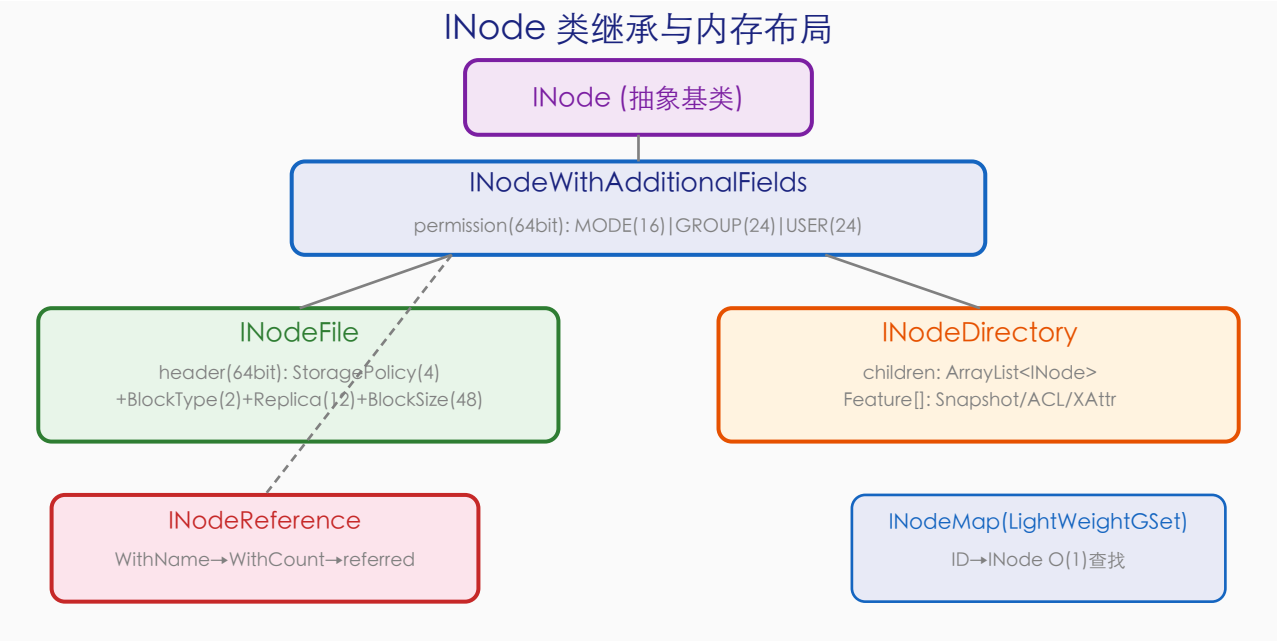


图5-1：INode类继承层次与内存布局

5.2 PermissionStatusFormat — 位压缩权限

字段	位范围	位宽	说明
MODE	bit 0~15	16bit	Unix文件权限(rwxrwxrwx+特殊位)
GROUP	bit 16~39	24bit	通过SerialNumberManager映射到组名
USER	bit 40~63	24bit	通过SerialNumberManager映射到用户名

SerialNumberManager：String→int双向映射，所有INode共享同一映射表。每个INode仅需8字节存储完整权限信息。

5.3 INodeFile — HeaderFormat位编码

字段	位范围	说明
PREFERRED_BLOCK_SIZE	低48bit	首选块大小(字节)
REPLICATION/EC_POLICY_ID	中间12bit	副本数或EC策略ID

字段	位范围	说明
BLOCK_LAYOUT	2bit	块布局(CONTIGUOUS/STRIPED)
STORAGE_POLICY_ID	高4bit	存储策略ID

5.4 INodeReference — 快照引用链

三层引用链设计解决快照+重命名场景：WithName（名称引用）→ WithCount（引用计数）→ referred（实际INode）。DstReference处理重命名目标。多路径通过WithCount共享底层INode。

5.5 INodeMap — O(1)全局查找

使用LightWeightGSet（开放寻址哈希集合）实现INode ID到INode对象O(1)查找。匿名INode查找机制避免额外Key对象分配。

六、HA 高可用机制 — 状态模式

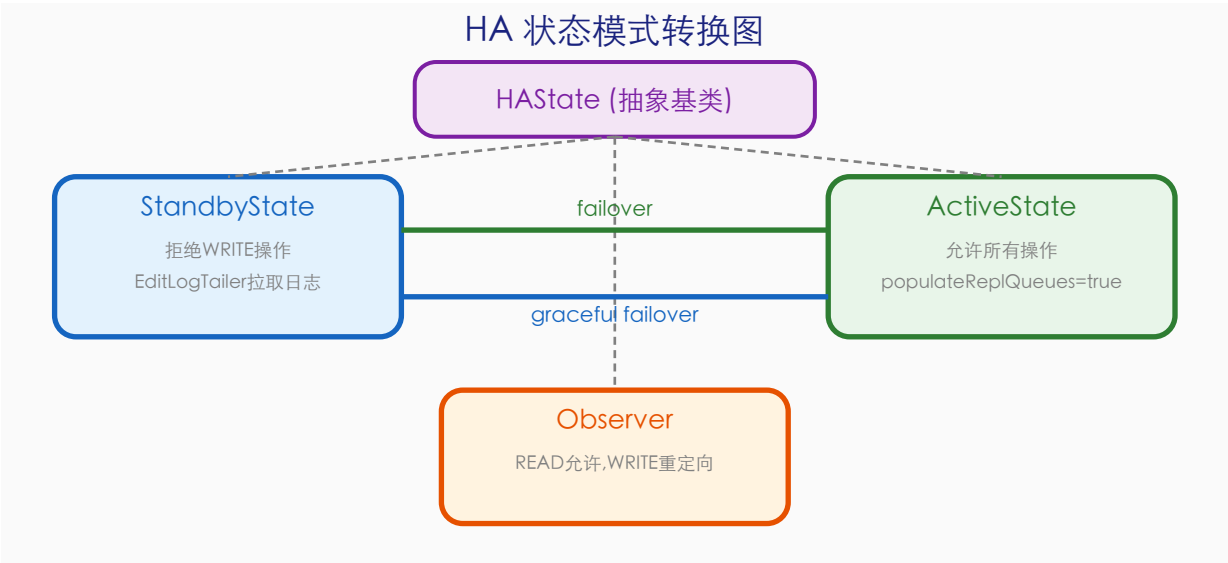


图6-1：HA状态机转换图

6.1 setStateInternal() — 模板方法

步骤	方法	说明	持锁
①	prepareToExitState()	准备退出当前状态	无锁
②	s.prepareToEnterState()	准备进入新状态	无锁
③	context.writeLock()	获取全局写锁	加锁
④	exitState()	退出当前状态（停服务）	持锁
⑤	context.setState(s)	原子设置新状态	持锁
⑥	s.enterState()	进入新状态（启服务）	持锁
⑦	updateLastHATransitionTime() ()	记录转换时间戳	持锁

6.2 三种状态行为差异

特性	ActiveState	StandbyState	Observer
允许操作	READ+WRITE	UNCHECKED+StaleREAD	READ(重定向WRITE)
填充副本队列	是	否	否
enterState()	startActiveServices()	startStandbyServices()	startStandbyServices()

特性	ActiveState	StandbyState	Observer
EditLogTailer	不需要	运行(拉取日志)	运行(拉取日志)
StandbyCheckpoint ointer	不需要	运行(定期合并)	运行

七、编辑日志与持久化机制

7.1 FSEditLog 状态机

状态	含义	允许操作
UNINITIALIZED	未初始化	→BETWEEN_LOG_SEGMENTS
BETWEEN_LOG_SEGMENTS	日志段间隔	→IN_SEGMENT或→OPEN_FOR_READING
IN_SEGMENT	正在写入日志段	logEdit()/logSync(); →BETWEEN_LOG_SEGMENTS
OPEN_FOR_READING	只读(Standby)	仅读取
CLOSED	已关闭	终态

7.2 logSync() — 批量同步

阶段	操作	持锁	说明
等待	while(mytxid>syncctxid && isSyncRunning) wait()	sync	等待其他线程同步完成
检查	if(mytxid<=syncctxid) return	sync	事务已被批量同步覆盖
交换	editLogStream.setReadyToFlush()	sync	交换双缓冲区A↔B
刷盘	logStream.flush()	无锁	实际I/O持久化(耗时)
更新	syncctxid=syncStart; notifyAll()	sync	更新已同步txid并唤醒

核心设计：双缓冲区让写入和刷盘并行；批量同步减少I/O；Fail-Fast策略防数据不一致；JournalSet多副本写入提供冗余。

7.3 FSImage — 检查点持久化

启动时加载最新FSImage→重放EditLog恢复命名空间。saveNamespace()将内存序列化到fsimage_txid文件。StandbyCheckpoint定期合并FSImage+EditLog生成新Checkpoint并上传到Active NN。HA场景延迟创建VERSION文件。

八、FSNamesystemLock — 读写锁机制

FSNamesystemLock（386行）封装ReentrantReadWriteLock，增加丰富的监控诊断：

特性	实现	说明
读锁计时	ThreadLocal readLockHeldTimeMs	每个读锁线程的持有时间
写锁计时	writeLockHeldTimeMs	当前写锁持有时间
慢锁日志	readLockReportingThresholdMs	超阈值输出警告日志
最长读锁	longestReadLockHeldInfo(AtomicReference)	CAS无锁方式记录历史最长读锁
日志限流	LogThrottlingHelper	限制慢锁日志输出频率
锁持有者	writeLockOwnerThread	记录写锁持有者线程名

九、核心流程时序分析

9.1 文件创建 startFileInt() 详细流程

1. 路径校验：DFSUtil.isValidName(src) + 保留名检查
2. 副本策略：SHOULD_REPLICATE → verifyReplication() / 否则检查EC策略
3. 获取写锁：writeLock(RwLockMode.FS)
4. HA+SafeMode检查：checkOperation(WRITE) + checkNameNodeSafeMode()
5. 路径解析：FSDirWriteFileOp.resolvePathForStartFile()
6. 加密区域：释放锁→生成EDEK→重新获取锁→重新解析路径
7. 目录树修改：dir.writeLock() → FSDirWriteFileOp.startFile() → dir.writeUnlock()
8. 日志同步：释放锁后 → getEditLog().logSync()

9.2 DataNode心跳处理 — BPServiceActor.offerService()

每轮循环（~3秒）：① 发送心跳sendHeartBeat() → ② 更新HA状态 → ③ 处理NN命令（KeyUpdate优先入队）→ ④ 发送IBR增量块报告 → ⑤ 全量块报告（默认6小时，租约机制）→ ⑥ 缓存报告 → ⑦ 等待

9.3 块报告处理 — processReport()

持有全局写锁(RwLockMode.GLOBAL)。首次报告processFirstBlockReport()高效处理加速启动。后续报告通过reportDiff()计算五个差异队列：toAdd/toRemove/toInvalidate/toCorrupt/toUC。

十、设计模式总结

设计模式	应用场景	关键类	设计意图
门面模式	NameNode 核心入口	FSNamesystem	统一入口协调FSDirectory/BlockManager/EditLog等子系统
状态模式	HA状态管理	HActiveState/StandbyState	Active/Standby/Observer行为封装在独立状态类中
模板方法	HA状态转换	HActiveState.setStateInternal()	定义prepareExit→exit→setState→enter的固定步骤
策略模式	块副本放置	BlockPlacementPolicy	副本放置算法可插拔，配置切换
组合模式	文件系统树	INode/INodeFile/INodeDirectory	统一文件/目录操作接口，支持递归
装饰器模式	INode特性扩展	Feature接口(Acl/XAttr...)	Feature[] 动态扩展INode功能
观察者模式	编辑日志多副本	JournalSet+JournalManager[]	logEdit广播到所有Journal
双缓冲区	编辑日志同步	FSEditLog	写入和刷盘并行，批量同步提升吞吐
租约模式	文件写入控制	LeaseManager	软/硬限制控制写入并发
写时复制	快照管理	DirectoryWithSnapshotFeature	快照存在时记录diff，不复制整树

表10-1：NameNode核心设计模式一览

十一、关键场景调用链

11.1 文件创建

```
Client.create() →  
DFSClient →  
NameNodeRpcServer.create() →  
FSNamesystem.startFile() →  
startFileInt() →  
FSDirWriteFileOp.startFile() →  
FSEditLog.logOpenFile() →  
logSync()
```

11.2 块分配

```
Client.addBlock() →  
NameNodeRpcServer.addBlock() →  
FSNamesystem.getAdditionalBlock() →  
BlockManager.chooseTarget4NewBlock() →  
BlockPlacementPolicyDefault.chooseTargetInOrder()
```

11.3 心跳处理

```
BPSERVICEActor.offerService() →  
sendHeartBeat() →  
DatanodeProtocol.sendHeartbeat() →  
NameNodeRpcServer.sendHeartbeat() →  
FSNamesystem.handleHeartbeat() →  
DatanodeManager.handleHeartbeat()
```

11.4 块报告

```
BPSERVICEActor.blockReport() →  
NameNodeRpcServer.blockReport() →  
BlockManager.processReport() →  
processFirstBlockReport() ■ reportDiff() →  
addStoredBlock()/removeStoredBlock()
```

11.5 HA Failover

```
ZKFC.becomeActive() →  
HASERVICEProtocol.transitionToActive() →  
NameNode.transitionToActive() →  
HAState.setStateInternal(ACTIVE) →  
StandbyState.exitState() →  
ActiveState.enterState()
```

11.6 快照创建

```
Client.createSnapshot() →  
NameNodeRpcServer.createSnapshot() →  
FSNamesystem.createSnapshot() →  
SnapshotManager.createSnapshot() →  
INodeDirectory.addSnapshot()
```

11.7 租约恢复

```
LeaseManager.checkLeases() →  
■■■■■■■■ →  
internalReleaseLease() →  
FSNamesystem.recoverLeaseInternal() →  
BlockManager.commitOrCompleteLastBlock()
```

十二、子模块概览

子目录	文件数	核心职责	关键类
ha/	~9	HA状态机、EditLogTailer、StandbyCheckpointeer	HAState, ActiveState, StandbyState, EditLogTailer
snapshot/	~19	目录快照管理、快照差异计算	SnapshotManager, DirectorySnapshottableFeature, SnapshotDiffReport
metrics/	~4	NameNode指标收集和上报	NameNodeMetrics, FSNamesystemMBean
startupprogress/	~11	启动进度追踪和报告	StartupProgress, Phase, Step, Status
sps/	~12	存储策略满足器（数据迁移）	StoragePolicySatisfier, BlockMoveTaskHandler
fgl/	~4	细粒度锁新方案（性能优化）	FineGrainedFSNamesystemLock
top/	~7	操作审计和Top操作统计	TopAuditLogger, TopConf
visitor/	~4	FSImage访问者模式	ImageVisitor, TextWriterImageVisitor

十三、核心源文件清单

源文件	大小	核心职责
NameNode.java	110KB	HDFS主入口，HA状态管理，生命周期控制
FSNamesystem.java	341KB	命名空间核心状态机（模块最大文件），门面模式
FSDirectory.java	75KB	内存INode目录树管理，配额/ACL/XAttr
FSEditLog.java	64KB	编辑日志管理（双缓冲区+批量同步+状态机）
FSImage.java	60KB	文件系统镜像持久化（Checkpoint）
NameNodeRpcServer.java	98KB	RPC三通道服务端，实现NamenodeProtocols
FSNamesystemLock.java	~5KB	读写锁封装，慢锁监控和诊断
INode.java	39KB	文件/目录抽象基类（快照感知操作）
INodeFile.java	42KB	文件INode（64bit header位编码）
INodeDirectory.java	35KB	目录INode（children列表，快照特性）
INodeReference.java	25KB	快照引用链（WithName/WithCount/DstReference）
INodeMap.java	~4KB	LightWeightGSet全局ID→INode映射
LeaseManager.java	22KB	文件租约管理（软/硬限制，租约恢复）
NNStorage.java	30KB	NameNode物理存储目录管理
JournalSet.java	~15KB	多Journal管理器集合
HAState.java	5KB	HA状态机基类（状态模式+模板方法）
ActiveState.java	2.4KB	Active状态实现
StandbyState.java	4KB	Standby/Observer状态实现
SecondaryNameNode.java	35KB	辅助NameNode（CheckpointNode）
CacheManager.java	30KB	DataNode缓存指令管理
EncryptionZoneManager.java	12KB	加密区域管理
SnapshotManager.java	31KB	快照全局管理器

表13-1：NameNode核心源文件清单（22个关键文件）

十四、设计亮点与总结

14.1 设计亮点

- 极致内存优化：PermissionStatusFormat 64bit编码权限、INodeFile header 64bit编码属性、SerialNumberManager字符串去重，每个INode节省数十字节
- 双缓冲区+批量同步：FSEditLog写入和刷盘并行，多线程edit一次flush持久化，I/O吞吐量极高
- 三通道RPC隔离：client/service/lifeline三个RPC Server互不阻塞，lifeline确保HA健康检查不受业务影响
- 状态模式HA：Active/Standby/Observer行为封装在独立类中，setStateInternal()模板方法确保转换原子性
- 写时复制快照：DiffList记录增量变更而非全量复制，内存开销与变更量成正比
- FSNamesystemLock诊断：CAS无锁最长锁记录、LogThrottlingHelper日志限流、锁持有者追踪
- 分层门面架构：FSNamesystem作为统一入口，内部子系统高内聚低耦合

14.2 质量评估

评估维度	评分	说明
架构分层	★★★★★	四层架构清晰，RPC/业务/数据/HA分离彻底
设计模式运用	★★★★★	门面/状态/模板/策略/组合/装饰/观察者等模式运用精准
内存效率	★★★★★	位编码/序号映射/写时复制等优化极致
可扩展性	★★★★☆	块放置策略可插拔，HA状态可扩展，但FSNamesystem过大
并发控制	★★★★☆	读写锁+双缓冲区高效，但全局写锁是瓶颈（fgl模块正在优化）
代码可维护性	★★★☆☆	FSNamesystem 341KB过大，需进一步拆分

— 文档结束 —